

# Exploring Numerical Algorithms: Root Finding and Integration Techniques

**Module:** 4MM013: Computational Mathematics

**Assignment Report:** Portfolio

**Date of Submission:**

25th May, 2026

**Name:** Saurabh Kumar Yadav

**Group:** 3

**Student number:** 2592143

## **Introduction:**

The report demonstrates four numerical methods commonly used when an exact answer is difficult to obtain or less useful than a good approximation. The root-finding part compares Bisection Method and Newton's Method and the integration part compares Midpoint Rule and Trapezoidal Rule. The algorithms have been implemented in Python and tested on selected functions. The results have been compared with reference or analytical values. The main aim was not only to get the final numbers but to understand how the methods behave when the starting values, intervals and number of sub intervals are varied.

## **1) Root Finding Method:**

### **A. Bisection Method:**

It starts with two endpoints,  $a$  and  $b$ , such that  $f(a)$  and  $f(b)$  have opposite signs. Then it divides the interval and takes the half in which the sign change still occurs. Doing this repeatedly again reduces the interval smaller and smaller until the midpoint is close enough to the root.

#### **Advantages:**

- It is very reliable, if one chooses a valid interval.
- It does not require the derivative of the function.
- All iterations are easy to understand and implement in Python

#### **Disadvantages:**

- It could be slower, since it only halves the interval each time.
- it requires an interval where the function values have opposite signs.
- It might not find roots if the graph just touches the x-axis and doesn't cross it.

### **B. Newton's Method:**

Newton's Method uses a starting guess and the slope of the function to quickly move toward a root. At each step it draws a tangent line at the current point and uses the x-intercept of the tangent as the next guess. If the initial guess is reasonable and the derivative is not too small, the method usually converges very quickly. However, it is more sensitive than bisection to a bad starting value which can cause the computation to diverge from the desired root.

### Advantages:

- It usually converges in fewer iterations than the bisection method.
- It is efficient for smooth functions when a good first guess is given.
- It provides very precise roots with a small amount of computation.

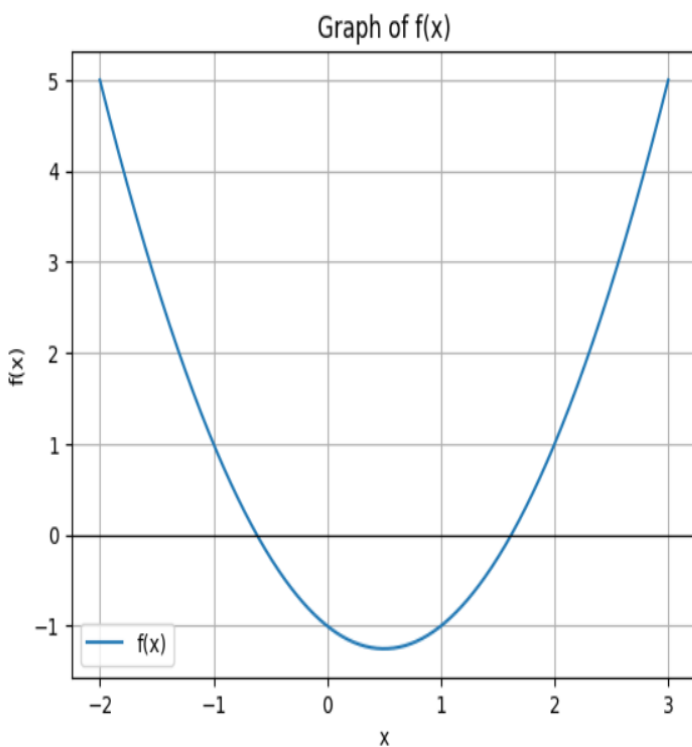
### Disadvantages:

- It requires derivative of the function.
- If the derivative is close to zero, it can become unstable or break down.
- The final root is highly dependent on the initial guess.

### Results and Graphs:

#### Bisection Method:

$$f(x) = x^2 - x - 1$$



$$f(x) = x^3 - x^2 - 2x + 1$$

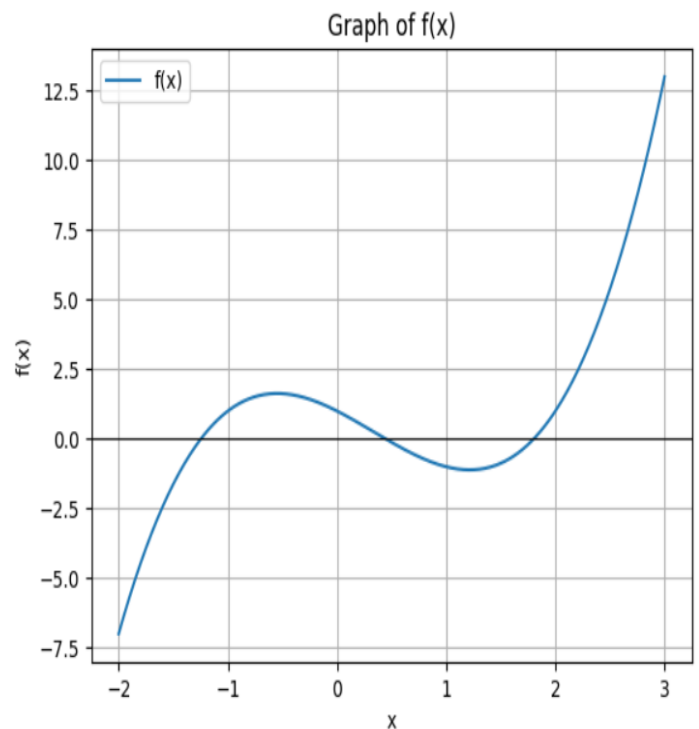
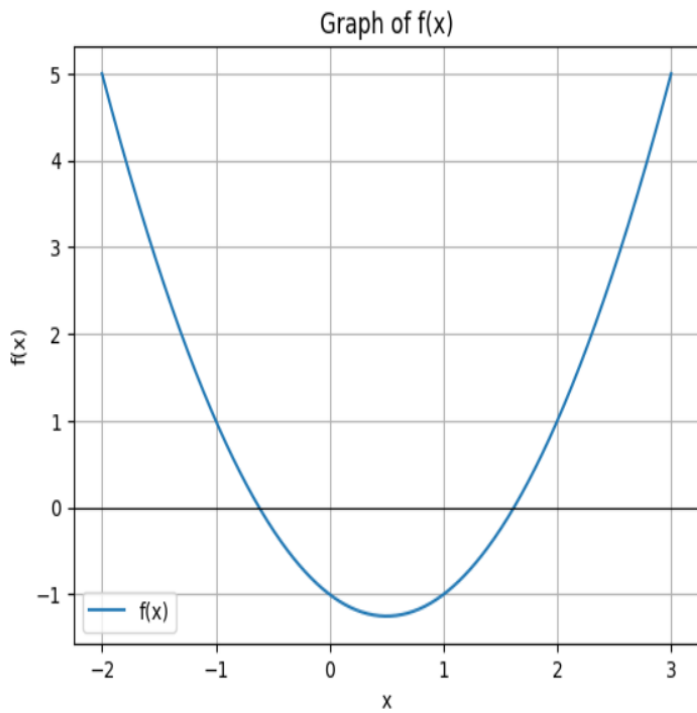


Fig: Graph of Bisection Method

### Newton's Method:

$$f(x) = x^2 - x - 1$$



$$f(x) = x^3 - x^2 - 2x + 1$$

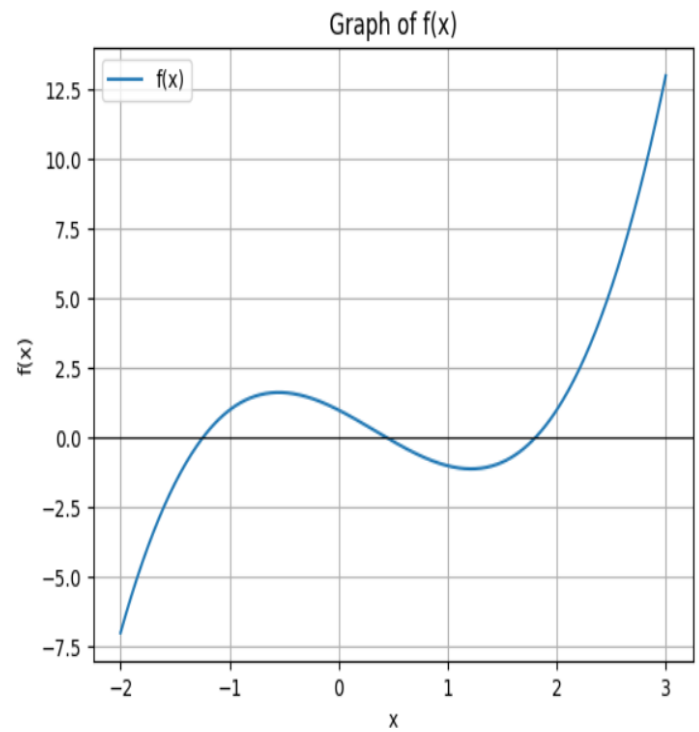


Fig: Graph of Newton's Method

| For Function 1          |                          |                          |                           |
|-------------------------|--------------------------|--------------------------|---------------------------|
| Bisection Method        |                          | Newton's Method          |                           |
| [1, 2]<br>21 iterations | [-1, 0]<br>21 iterations | x0 = 1.5<br>3 iterations | x0 = -0.5<br>3 iterations |
| Root1 = 1.61803389      | Root2 = -0.61803389      | Root1 = 1.61803399       | Root2 = -0.61803399       |
| SciPy Method            |                          |                          |                           |
| x0 = 1.5                |                          | x0 = -0.5                |                           |
| Root1 = 1.61803399      |                          | Root2 = -0.61803399      |                           |

| For Function 2            |                         |                         |                           |                          |                          |
|---------------------------|-------------------------|-------------------------|---------------------------|--------------------------|--------------------------|
| Bisection Method          |                         |                         | Newton's Method           |                          |                          |
| [-2, -1]<br>21 iterations | [0, 1]<br>21 iterations | [1, 2]<br>21 iterations | x0 = -1.5<br>4 iterations | x0 = 0.5<br>2 iterations | x0 = 1.5<br>5 iterations |
| Root1 = -1.24697924       | Root2 = 0.44504213      | Root3 = 1.80193758      | Root1 = -1.24697960       | Root2 = 0.44504182       | Root3 = 1.80193774       |
| SciPy Method              |                         |                         |                           |                          |                          |
| x0 = -1.5                 |                         | x0 = 0.5                |                           | x0 = 1.5                 |                          |

|                     |                    |                    |
|---------------------|--------------------|--------------------|
| Root1 = -1.24697960 | Root2 = 0.44504187 | Root3 = 1.80193774 |
|---------------------|--------------------|--------------------|

The results show that both methods arrived at the same roots as the reference values. Bisection took more iterations but was predictable with the intervals being chosen around sign changes. Newton's method was more efficient, particularly for the quadratic function, but it depended on the choice of initial guesses near the roots.

## 2) **Numerical Integration Method:**

### **C. Midpoint rule:**

The Midpoint Rule approximates the area under a curve by dividing the interval into equal parts and using the height of the function at the midpoint of each subinterval. This often gives a good estimate because the midpoint can cancel out the overestimation and underestimation that occurs inside a small interval.

#### **Advantages:**

- It is simple and usually more accurate than just using left or right endpoints.
- Obviously, the accuracy is improved with the increase of the number of subintervals.
- It is good for smooth functions like the ones tested in this report.

#### **Disadvantages:**

- It is still an approximation so there is still some error unless N is very large.
- Needs the values of the function at many midpoints.
- For functions that vary sharply a small N may give a poor estimate.

### **D. Trapezoid Rule:**

The Trapezoidal Rule approximates the area under the curve by connecting adjacent points with straight lines. Each small area is treated as a trapezium and all those areas are added up. The method is intuitive because it follows the general shape of the graph. However, for curved functions it might over- or underestimate depending on the direction of curvature.

### Advantages:

- Easy to code and explain.
- It uses the end points of each subinterval. So, it traces the general shape of the graph.
- The accuracy increases as the interval is divided into more subintervals.

### Disadvantages:

- For highly curved functions, the approximation may be less accurate when N is small.
- It assumes a linear connection between points, while the actual function may curve.
- The error is considerably larger for low N in the exponential function.

## Results and Graphs:

### Midpoint Approx Method and Trapezoidal Approx Method

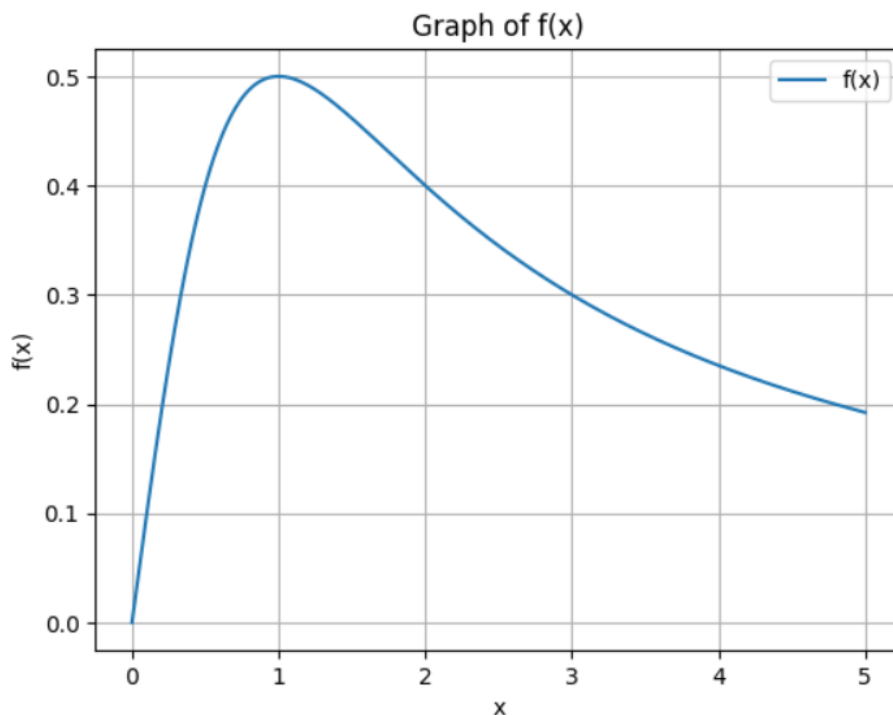


Fig: Graph of function 1:  $f(x) = x / (x^2 + 1)$

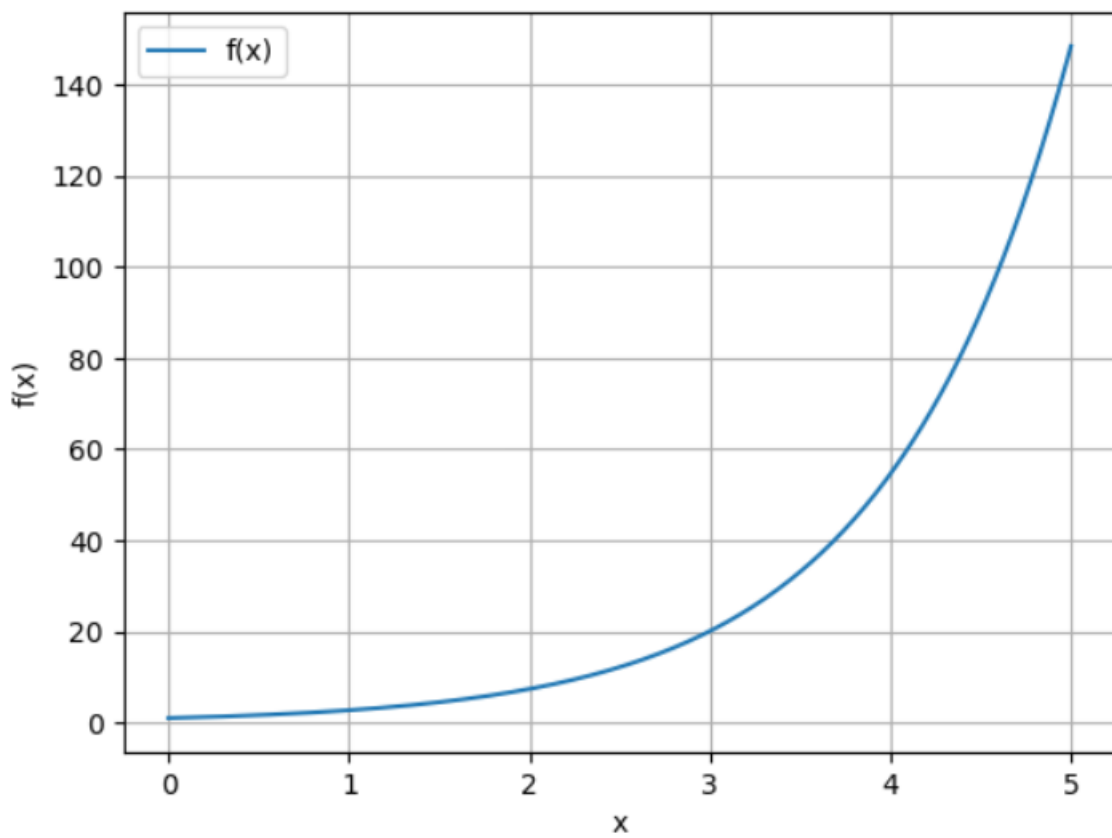


Fig: Graph of function 2:  $f(x) = e^x$



| N              | Midpoint<br>Approximation ( $M_N$ ) | Trapezoidal Rule<br>( $T_N$ ) | Analytical (I) | Abs<br>Error<br>$ M_N - I $ | Abs<br>Error<br>$ T_N - I $ |
|----------------|-------------------------------------|-------------------------------|----------------|-----------------------------|-----------------------------|
| For Function 1 |                                     |                               |                |                             |                             |
| 10             | 1.640378                            | 1.606865                      | 1.629048       | 0.01132983                  | 0.02218344                  |
| 30             | 1.630252                            | 1.626645                      | 1.629048       | 0.00120420                  | 0.00240351                  |
| 50             | 1.629480                            | 1.628185                      | 1.629048       | 0.00043219                  | 0.00086376                  |
| 100            | 1.629156                            | 1.628832                      | 1.629048       | 0.00010791                  | 0.00021578                  |
| 500            | 1.629053                            | 1.629040                      | 1.629048       | 0.00000431                  | 0.00000863                  |
| For Function 2 |                                     |                               |                |                             |                             |
| 10             | 145.888729                          | 150.471546                    | 147.413159     | 1.52443031                  | 3.05838690                  |
| 30             | 147.242680                          | 147.754235                    | 147.413159     | 0.17047895                  | 0.34107629                  |
| 50             | 147.351755                          | 147.535983                    | 147.413159     | 0.06140424                  | 0.12282383                  |
| 100            | 147.397805                          | 147.443869                    | 147.413159     | 0.01535442                  | 0.03070980                  |
| 500            | 147.412545                          | 147.414388                    | 147.413159     | 0.00061422                  | 0.00122844                  |

For both integration examples, the absolute error became smaller as  $N$  increased from 10 to 500. This confirms the expected behaviour: using more subintervals gives the approximation more information about the curve. In the first function, both rules became very accurate quickly. In the exponential example, the Trapezoidal Rule started with a larger error because  $e^x$  bends upward strongly, but it also improved steadily as  $N$  increased.

## **Conclusion:**

Overall, the investigation showed that no single numerical method is automatically the best in every situation. For root finding, the Bisection Method was slower but very trustworthy once the interval contained a sign change. Newton's Method was faster, but it needed a useful initial guess and a derivative that did not become too small. For numerical integration, both the Midpoint and Trapezoidal rules improved when the number of subintervals was increased. The main lesson I took from the work is that numerical methods should be judged by both accuracy and behaviour: a method should not only give a good answer, but should also be understandable and stable enough to trust for the problem being solved.

## **I. References**

### **Bisection Method:**

^

```
--- Function 1: f(x) = x^2 - x - 1 ---
1st root: interval = [1, 2], iterations = 21, Bisection Method = 1.61803389.
1st root found by SciPy = 1.61803399
2nd root: interval = [-1, 0], iterations = 21, Bisection Method = -0.61803389.
2nd root found by SciPy = -0.61803399

--- Function 2: f(x) = x^3 - x^2 - 2x + 1 ---
1st root: interval = [-2, -1], iterations = 21, Bisection Method = -1.24697924.
1st root found by SciPy = -1.24697960
2nd root: interval = [0, 1], iterations = 21, Bisection Method = 0.44504213.
2nd root found by SciPy = 0.44504187
3rd root: interval = [1, 2], iterations = 21, Bisection Method = 1.80193758.
3rd root found by SciPy = 1.80193774
```

### **Newton's Method:**

^

```
--- Function 1: f(x) = x^2 - x - 1 ---
1st root: initial guess x0 = 1.5, iterations = 3, Newton's Method = 1.61803399.
1st root found by SciPy = 1.61803399
2nd root: initial guess x0 = -0.5, iterations = 3, Newton's Method = -0.61803399.
2nd root found by SciPy = -0.61803399
```

x

```
--- Function 2: f(x) = x^3 - x^2 - 2x + 1 ---  
1st root: initial guess x0 = -1.5, iterations = 4, Newton's Method = -1.24697960.  
1st root found by SciPy = -1.24697960  
2nd root: initial guess x0 = 0.5, iterations = 2, Newton's Method = 0.44504182.  
2nd root found by SciPy = 0.44504187  
3rd root: initial guess x0 = 1.5, iterations = 5, Newton's Method = 1.80193774.  
3rd root found by SciPy = 1.80193774
```

---

## Midpoint Approx Method

```
--- Function 1: f(x) = x/(x^2 + 1) ---
```

```
N = 10  
Subinterval width = 0.500000  
Midpoint Approximation for 1st Function = 1.640378  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.01132983
```

```
N = 30  
Subinterval width = 0.166667  
Midpoint Approximation for 1st Function = 1.630252  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.00120420
```

```
N = 50  
Subinterval width = 0.100000  
Midpoint Approximation for 1st Function = 1.629480  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.00043219
```

```
N = 100  
Subinterval width = 0.050000  
Midpoint Approximation for 1st Function = 1.629156  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.00010791
```

```
N = 500  
Subinterval width = 0.010000  
Midpoint Approximation for 1st Function = 1.629053  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.00000431
```

--- Function 2:  $f(x) = e^x$  ---

N = 10

Subinterval width = 0.500000

Midpoint Approximation for 2nd Function = 145.888729

Actual Value for 2nd Function = 147.413159

Absolute error between the above methods = 1.52443031

N = 30

Subinterval width = 0.166667

Midpoint Approximation for 2nd Function = 147.242680

Actual Value for 2nd Function = 147.413159

Absolute error between the above methods = 0.17047895

N = 50

Subinterval width = 0.100000

Midpoint Approximation for 2nd Function = 147.351755

Actual Value for 2nd Function = 147.413159

Absolute error between the above methods = 0.06140424

N = 100

Subinterval width = 0.050000

Midpoint Approximation for 2nd Function = 147.397805

Actual Value for 2nd Function = 147.413159

Absolute error between the above methods = 0.01535442

N = 500

Subinterval width = 0.010000

Midpoint Approximation for 2nd Function = 147.412545

Actual Value for 2nd Function = 147.413159

Absolute error between the above methods = 0.00061422

## Trapezoidal Approx Method

--- Function 1:  $f(x) = x/(x^2 + 1)$  ---

N = 10  
Subinterval width = 0.500000  
Trapezoidal Approximation for 1st Function = 1.606865  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.02218344

N = 30  
Subinterval width = 0.166667  
Trapezoidal Approximation for 1st Function = 1.626645  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.00240351

N = 50  
Subinterval width = 0.100000  
Trapezoidal Approximation for 1st Function = 1.628185  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.00086376

N = 100  
Subinterval width = 0.050000  
Trapezoidal Approximation for 1st Function = 1.628832  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.00021578

N = 500  
Subinterval width = 0.010000  
Trapezoidal Approximation for 1st Function = 1.629040  
Actual Value for 1st Function = 1.629048  
Absolute error between the above methods = 0.00000863

---

--- Function 2:  $f(x) = e^x$  ---

N = 10  
Subinterval width = 0.500000  
Trapezoidal Approximation for 2nd Function = 150.471546  
Actual Value for 2nd Function = 147.413159  
Absolute error between the above methods = 3.05838690

N = 30  
Subinterval width = 0.166667  
Trapezoidal Approximation for 2nd Function = 147.754235  
Actual Value for 2nd Function = 147.413159  
Absolute error between the above methods = 0.34107629

N = 50  
Subinterval width = 0.100000  
Trapezoidal Approximation for 2nd Function = 147.535983  
Actual Value for 2nd Function = 147.413159  
Absolute error between the above methods = 0.12282383

N = 100  
Subinterval width = 0.050000  
Trapezoidal Approximation for 2nd Function = 147.443869  
Actual Value for 2nd Function = 147.413159  
Absolute error between the above methods = 0.03070980

N = 500  
Subinterval width = 0.010000  
Trapezoidal Approximation for 2nd Function = 147.414388  
Actual Value for 2nd Function = 147.413159  
Absolute error between the above methods = 0.00122844

---